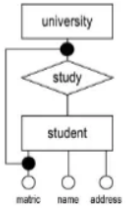
ER Diagram

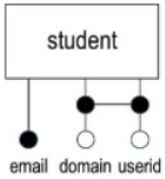


Partial key:
- some entities can only be identified within the scope of a relationship with another entity set (weak entity)
- matric numbers are given by universities. same number can be used by different universities
- university is a dominant entity
- student is a weak entity

Cardinality:
(1, x): mandatory participation
(0, x): optional participation
(x, 1): one-to-one if (x, 1) for all entities involved
(x, 1): one-to-many if (x, 1) for one but (x, n) or (x, y) for y > 1 for other
(x, n): many-to-many

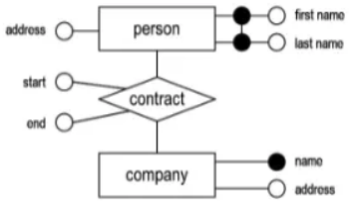
Weak entities can only be defined for a participation constrained by (1,1) cardinalities

Cardinality Rules:
1. Value sets are mapped to domains
2. Entity sets are mapped to relations. Candidate keys are mapped to pkeys and unique non null




```
CREATE TABLE student (
    email VARCHAR(64) PRIMARY KEY,
    domain VARCHAR(12) NOT NULL,
    userid VARCHAR(50) NOT NULL,
    UNIQUE (domain, userid)
);
```

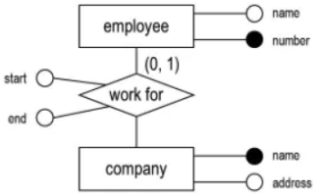
3. Relationship sets are mapped to relations. The keys are the keys of the participating entities.




```
CREATE TABLE contract(
    start DATE NOT NULL,
    end DATE NOT NULL,
    first_name VARCHAR(32),
    last_name VARCHAR(32),
    name VARCHAR(64),
    PRIMARY KEY (first_name, last_name, name),
    FOREIGN KEY (first_name , last_name)
        REFERENCES person(first_name, last_name),
    FOREIGN KEY (name) REFERENCES company(name)
);
```

Cardinality Exceptions

1. One-to-Many

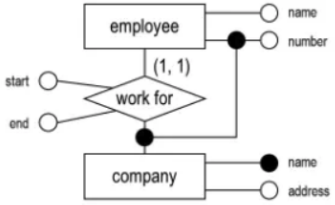



```
CREATE TABLE work_for(
    start DATE NOT NULL,
    end DATE NOT NULL,
    enumber CHAR(8) PRIMARY KEY,
    -- Restrict pkey
    cname VARCHAR(32) NOT NULL,
    -- NOT NULL to ensure existence
    FOREIGN KEY (enumber)
        REFERENCES employee(number),
     FOREIGN KEY (cname) REFERENCES
company(name)
);
```

2. One-to-One

```
CREATE TABLE employee_work_for(
    start DATE NOT NULL,
    end DATE NOT NULL,
    enumber CHAR(8) PRIMARY KEY,
    cname VARCHAR(32) NOT NULL,
    FOREIGN KEY (cname)
        REFERENCES company(name));
```

3. Weak entity




```
CREATE TABLE employee_work_for(
    start DATE NOT NULL,
    end DATE NOT NULL,
    enumber CHAR(8),
    ename VARCHAR(31) NOT NULL
    cname VARCHAR(32) NOT NULL,
    PRIMARY KEY (enumber, cname),
    FOREIGN KEY (cname)
        REFERENCES company(name));
```

Relational Algebra

π[rname](σ[(pizza = 'Veggie' ∧ price < 14) ∨ (rname = 'Sizzle Grill')](sells))	SELECT DISTINCT s.rname FROM sells s WHERE (s.pizza = 'Veggie' AND s.price < 14) OR (s.rname = 'Sizzle Grill')
π[r.attr](σ[c](ρ(rel, r)))	SELECT DISTINCT r.attr FROM rel r WHERE c;
Q1 := π[pizza](σ[rname = 'Bella Italia'](sells)) Q2 := π[pizza](σ[rname = 'Desert Diner'](sells)) Q1 − Q2	SELECT s.pizza FROM sells s WHERE s.rname = 'Bella Italia' EXCEPT SELECT s.pizza FROM sells s WHERE s.rname = 'Desert Diner';
π[c.cname, r.rname](σ[c.area = r.area](ρ(customer, c) × ρ(restaurant, r)))	SELECT c.cname, r.rname FROM customer c, restaurant r WHERE c.area = r.area;
π[c.cname, r.rname](ρ(customer, c) ⋈[c.area = r.area] ρ(restaurant, r))	SELECT c.cname, r.rname FROM restaurant r, customer c WHERE r.area = s.area;

Operation	Symbol	Operation	Symbol	Operation	Symbol	Operation	Symbol
Selection	σ	Projection	π	Renaming	ρ	Cross Product	×
Conjunction	∧	Disjunction	∨	Negation	¬	Join	⋈
Union	∪	Intersection	∩	Difference	−	Outer Joins	⟕ ⟖ ⟗

Stored Procedures

Statement level interface: host lang + SQL

```
// Declare cursor
EXEC SQL DECLARE cursor CURSOR FOR
SELECT prodName
FROM Sells
WHERE supplName = :supplier;
// Open cursor
EXEC SQL OPEN cursor; EXEC SQL
WHENEVER NOT FOUND DO BREAK;
for (;;) {
     EXEC SQL FETCH NEXT FROM cursor
INTO :v_product;
        printf(">>>  product:  %s\n",
v_product);
} // close cursor, disconnect
```

```
import psycopg # Host language
library

# Connect to db
conn = psycopy.connect("host= ... ")

# Create cursor
cursor = conn.cursor()

# Open cursor by executing query
cursor.execute("SELECT supplName from
Sells")

while True:
        row = cursor.fetchone()
        if row is None:
                break
        print(f">>>Supplier:
{row[0]})

# Cleanup
cursor.close()
conn.commit()
conn.close()
```

Prepared SQL

```
EXEC SQL BEGIN DECLARE SECTION;
        const char *query = "INSERT INTO test VALUES(?, ?);";
EXEC SQL END DECLARE SECTION;
EXEC SQL CONNECT @localhost USER john;
EXEC SQL PREPARE stmt FROM :query; // parsed, compiled by databse ONCE only
EXEC SQL EXECUTE stmt USING 42, 'foobar';
EXEC SQL DEALLOCATE PREPARE stmt; – deallocate
```

Functions

```
CREATE OR REPLACE FUNCTION <name> (<param> <type>, <param> <type>, ... )
RETURN <type> AS $$
        <code>
$$ LANGUAGE <language>
```

```
-- Finding students with
highest mark
CREATE OR REPLACE FUNCTION
topStudents()
RETURNS SETOF Scores AS $$
    SELECT * FROM Scores
    WHERE Mark = (
        SELECT MAX(Mark)
        FROM Scores);
$$ LANGUAGE sql;
```

```
-- Finding highest mark and its count --
if we use RECORD, we must have at least 2
OUT params
CREATE OR REPLACE FUNCTION
topMarkCount(OUT Mark INT, OUT Count INT)
RETURNS RECORD AS $$
    SELECT Mark, COUNT(*) FROM Scores
    WHERE Mark = (
        SELECT MAX(Mark) FROM Scores);
    GROUP BY Mark;
$$ LANGUAGE sql;
```

```
-- Finding all distinct marks
and their count
CREATE OR REPLACE FUNCTION
markCounts(OUT Mark INT, OUT
Count INT)
RETURN SETOF RECORD AS $$
    SELECT Mark, COUNT(*)
    FROM Scores
    GROUP BY Mark;
$$ LANGUAGE sql;

CREATE OR REPLACE FUNCTION
markCounts()
RETURNS TABLE(Mark INT, Count
INT) AS $$
    SELECT Mark, COUNT(*)
    FROM Scores
    GROUP BY Mark;
$$ LANGUAGE sql;
```

```
-- Transfer money from A to B -- no
return value: VOID or PROCEDURE
CREATE OR REPLACE FUNCTION transfer(frAcc
TEXT, toAcc TEXT, amount INT)
RETURN VOID AS $$
    UPDATE Acct SET balance = balance -
amount
    WHERE name = frAcc;
    UPDATE Acct SET balance = balance +
amount
    WHERE name = toAcc; $
$ LANGUAGE sql;
SELECT tranfer('Alice', 'Bob', 100);

CREATE OR REPLACE PROCEDURE transfer
(frAcc TEXT, toAcc TEXT, amount INT) AS
$$
    UPDATE Acct SET balance = balance -
amount
    WHERE name = frAcc;
    UPDATE Acct SET balance = balance +
amount
    WHERE name = toAcc;
$$ LANGUAGE sql;
CALL transfer('Alice', 'Bob', 100);
```

Function must be declared to return smt (except void)
Procedure has no return but can use OUT param
Procedure can commit/roll back

```
s := 0; temp := 1;
WHILE temp ≤ x LOOP
    s := s + temp;
    temp := temp + 1;
END LOOP;

s := 0; temp := 1;
LOOP
    EXIT WHEN temp > x;
    s := s + temp;
    temp := temp + 1;
END LOOP;
```

```
prev := -1;
FOR r IN SELECT * FROM Scores ORDER BY Mark DESC
LOOP
    name_ := r.Name;
    mark_ := r.Mark;
    IF prev ≥ 0 THEN gap := prev - mark_; --
calculate difference
    ELSE gap := NULL;
    END IF;
    RETURN NEXT;
    prev := r.Mark; -- Store curr student's mark
for the next student's gap calculation
END LOOP;
```

Cursor movements

```
FETCH curs INTO r
FETCH PRIOR FROM curs INTO r -- fetch from prev row
FETCH FIRST FROM curs INTO r
FETCH LAST FROM curs INTO r
FETCH ABSOLUTE 3 FROM curs INTO r -- fetch the 3rd tuple
```

Triggers

	NULL tuple	non-NULL tuple t
BEFORE INSERT	No tuple inserted	t inserted
BEFORE UPDATE	No tuple updated	t will be the updated tuple
BEFORE DELETE	No deletion performed	deletion proceeds as normal

Trigger Options:
- INSERT ON
- DELETE ON
- UPDATE OF ON
- AFTER/BEFORE
- INSTEAD OF
- FOR EACH ROW
- FOR EACH STMT

Statement level trigger: ignores values returned by trigger func, RETURN NULL will not make the db omit the subseq. op. INSTEAD OF only allowed on row-level granularity
WHEN() rules: no SELECT, no OLD for INSERT, no NEW for DELETE, cannot be used for INSTEAD OF (eg `FOR EACH ROW WHEN (NEW.Name = 'Adi')`)

Deferred Trigger: check only at end of transaction
→ only work with AFTER and FOR EACH ROW

```
CREATE CONSTRAINT TRIGGER balance_check
AFTER INSERT OR UPDATE OR DELETE ON Account
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION check_balance();
```

Cardinality constraints

```
CREATE CONSTRAINT TRIGGER on_new_allocation
AFTER INSERT ON participates DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION on_insert_participates();

CREATE OR REPLACE FUNCTION on_insert_participates()
RETURNS TRIGGER AS $$
DECLARE team_size INT; num_projects INT
BEGIN
    SELECT COUNT(*) INTO team_size FROM participates
    WHERE pid = NEW.pid;
    SELECT COUNT(*) INTO num_projects FROM participates
    WHERE sid = NEW.sid;
    IF team_size < 2 OR team_size > 5 OR num_projects > 3 THEN
          RAISE EXCEPTION 'Invalid cardinality';
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

Order of op:
BEFORE statement lvl → BEFORE row-lvl → AFTER row lvl → AFTER statement lvl
Within each cat, triggers activated in alphabetic order
If BEFORE row-level trigger returns NULL, subsequent triggers on same row omitted

Functional Dependencies
eg NRIC → Name: NRIC decides Name
Armstrong's Axioms

- **Reflexivity**: a set of attributes → a subset of attributes
  - ABCD → ABC
  - ABCD → BCD
  - ABCD → AD
- **Augmentation**: if A → B, then AC → BC for any C
- **Transitivity**: if A → B and B → C, then A → C

Rule of Decomposition: if A → BC, then A → B and A → C
Rule of Union: if A → B and A → C, then A → BC
Closure: Let S = {A1, A2, …, An} be a set of attributes, S+ is the set of attributes that can be decided by A1, A2, …, An
Superkeys: set of attributes in table that decides all other attributes
Keys of a table: a superkey that is minimal (if we remove any attribute from superkey, it will not be a superkey anymore)

Finding keys of T:

- Given a table T(A, B, C, …) and a set of FDs on T
  → A table R(A, B, C) with A → B, B → C
- Consider every subset of attributes in T
  → subset of attributes, A, B, C, AB, BC, AC, ABC
- Derive the closure of each subset
  → {A}+ = {ABC}, {B}+ = {BC}, {C}+ = {C}
  → {AB}+ = {ABC}, {BC}+ = {BC}, {AB}+ = {ABC}
  → {ABC}+ = {ABC}
- Identify all superkeys based on closures
  → A, AB, AC, ABC
- Identify all keys from superkeys
  → A

Always check attribute sets first for the keys
If A doesnt appear in RHS of any functional dependency, A must be in every key
If an attribute appears in a key, it is a **prime attribute**

Normal Forms
Decomposed FD: an FD whose RHS has only 1 attribute (A → C, BCD → C)
Non-trivial FD: a FD whose RHS does not appear in LHS
We check NF against non-trivial decomposed FD

Finding non-trivial decomposed FD:

1. Consider all attribute subsets in R
2. Compute the closure of each subset
3. From each closure, remove trivial attributes
4. Derive non trivial and decomposed FDs from each closure

BCNF
R is in BCNF if every non trivial and decomposed FD has a superkey on its LHS

- A1A2…An → B then A1A2…An must be a superkey
- All attributes B can depend only on superkeys

Example: R(A, B, C) with A → B, B → A and B → C

- non trivial and decomposed FDs on R
  A → B, A → C, B → A, B → C, AB → C, AC → B, BC → A
- keys: A, B
- for each of the above FD, the LHS is a superkey → R satisfies BCNF

Checking if table is in BCNF
Example: R(A, B, C, D) with FDs AB → C, C → D and D → A
1. compute closure of each attribute subset
   - {A}+ = {A}
   - {B}+ = {B}
   - {C}+ = {ACD}
   - {D}+ = {AD}
2. check if there is a closure {A1A2…Ak}+ such that
   a. the closure contains some attribute not in {A1A2…Ak}
      - {C}+ = {ACD}. it contains more attributes than {C} does
   b. the closure does not contain all attribute in the table
      - {C}+ does not contain all attributes in R
3. if such a closure exists then R is NOT in BCNF

Decomposition algorithm
1. Find a subset X of the attributes in R such that its closure {X}+ contains more attributes than X does but does not contain all attributes in R
2. decompose R into 2 tables R1 and R2 such that
   a. R1 contains all attributes in {X}+
   b. R2 contains all attributes in X as well as the attributes not in {X}+
3. If R1/R2 is not in BCNF then decompose again
→ if a table only has 2 attributes, it must be in BCNF already

Finding FDs on decomposed table
1. Derive closures on R
2. Get the closures of attribute subsets that are attributes in R2
3. Derive closures of these attribute subsets on R
4. Project onto R2 by removing irrelevant attributes
→ BCNF may not always preserve FDs

Lossless Join Decomposition
→ Decomposition guarantees lossless join whenever the common attributes in R1 and R2 constitute a superkey of R1 or R2
E.g. R(A, B, C) decompose into R(A, B) and R2(B, C) with B being superkey of R1
→ Decomposition preserves all FDs iff S and S' are equivalent (S = set of FDs on given table, S' = set of FDs on decomposed tables)

3NF
Satisfying BCNF → satisfying 3NF
Violating 3NF → violating BCNF
A table satisfies 3NF iff for every non trivial and decomposed FD

- The LHS is a superkey OR
- RHS is a prime attribute (it appears in a key)

Example:
- non trivial and decomposed FDs: C → B, AC → B, AB → C
- keys: {AB}, {AC}
- C → B is OK as B is a prime attribute
- AC → B is OK as AC is a key of R
- AB → C is OK as AB is a key of R
- R is in 3NF

3NF check
1. derive keys of R
2. for each given FD, check if
   1. the LHS is a superkey or
   2. each attribute on RHS is a prime attribute
3. if all given FDs satisfy this condition, R is in 3NF

Minimal Basis
1: Every FD in MB can be derived from S, vice versa
2: Every FD in MB is a non trivial and decomposed FD
3: No FD in the MB is redundant ie no FD in MB can be derived from other FDs in MD
4: For each FD in MB, none of the attributes on the LHS is redundant ie if we remove an attribute from LHS, the resulting FD is a new FD that cannot be derived from original set of FDs

How to find MB
1. Transform the FDs such that each RHS contains only one attribute
   a. S = {A → BD, AB → C, C → D, BC → D}
   b. S = {A → B, A → D, AB → C, C → D, BC → D}
2. Remove redundant attributes on LHS of each FD
   a. Both AB → C and BC → D have more than 1 attribute on LHS
   b. is A redundant? B → C is not implied by S as {B}+ = {B} hence it is not implied by S hence A is not redundant
   c. B is redundant as {A}+ = {ABCD} so A → C is implied by S
   d. we can simplify AB → C to A → C
   e. Result: S = {A → B, A → D, A → C, C → D, BC → D}
   f. B is redundant in BC → D as C → D is implied by S
   g. Result: S = {A → B, A → D, A → C, C → D}
3. Remove redundant FDs
   a. A → D is redundant as it is implied by A → C and C → D
   b. Result: S = {A → B, A → C, C → D, BC → D}

Decomposition
- R(A,B,C,D) with S = {AB → C, C → A, B → D}
1. Derive a minimal basis of S
   a. a minimal basis of S is {AB → C, C → A, B → D}
2. In the minimal basis, combine the FDs whose LHS are the same
3. Create table for each FD remained
   a. R1(A,B,C), R2(C,A), R3(B,D)
4. If none of the tables contain a key of the original table R, create a table that contains a key of R (ensure lossless join decomposition)
5. Remove subsumed tables
   a. Do we need R2? everything in R2 is in R1 so we can remove it